25、编译器升级到7.5

全志V3S嵌入式驱动开发（编译器升级到7.5）

原创

嵌入式-老费Open: 7f185b501ef38a059b10d1b79d7cfdb9_MD5.png

已于 2023-08-29 15:12:15 修改

Open: 8a923f94c34ac151897c6be747452d7c_MD5.png

阅读量782 Open: 196806fd3bdd4f04e1a92a7697cf0354_MD5.png

收藏

Open: 95eb4f436541fdf1b2e1af49844ad0c3_MD5.png

点赞数 1

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

Open: 135c7190ea3ccdc82fa1f0c0a41d08f0_MD5.png

基于全志soc的linux嵌入式开发专栏收录该内容](https://blog.csdn.net/feixiaoxing/category_12270221.html "基于全志soc的linux嵌入式开发")

73 篇文章¥49.90¥99.00

已订阅8折续费

[

Open: d9b05f322fadabcd81dc6c5654189e59_MD5.png

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益Open: ddf4d655e9aa3806864ef10c7c4a060b_MD5.png

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

Open: 1e4395309f4514fed3e1f5e89b2a25e6_MD5.png

摘要 本文介绍了全志V3S驱动开发过程中，编译器从GCC 6.3.1升级到7.5的详细步骤，包括更新环境变量、重新编译uboot、内核、buildroot，并验证用户程序和驱动的兼容性，确保系统的正常运行。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing @163.com】

看过我们文章的朋友都知道，前面为了做v3s的驱动，对linux kernel进行了两次升级。第一次升级是从4.10.y升级到4.14.y，另外一次升级是从4.14.y升级到5.2.y。 **这中间最主要的原因，就是为了驱动适配的需要，更新的内核可以对硬件驱动有更好的支持，架构也更合理。**

如果说内核升级尚且可以理解，那为什么要对编译器升级呢？这里面，最重要的一点就是语言的升级。如果平时开发语言用到c++的同学应该也不陌生，这几年c++的标准一直在修改。这也导致了很多开源软件的写法也在同步修改。 **所以就有可能出现这样一种状况，那就是用老的编译器无法编译新的开源代码，这就是非常遗憾的一件事情。** 所以为了解决这一问题，我们也同样对编译器进行了升级，从6.3.1升级到7.5。

1、选择什么样的编译器

编译器最好和kernel比较匹配，比如kernel 5.2.y是在2019年发布的，所以我们可以选择在2019年release出来的gcc交叉编译器，比如7.5，

```
https://releases.linaro.org/components/toolchain/binaries/7.5-2019.12/arm-linux-gnueabihf/
```

2、更新环境变量

如果之前的编译器路径已经写到了.bashrc文件当中，那我们只需要解压一下编译器包，修改一下路径即可。因为编译器还都是arm-linux-gnueabihf-这样的前缀，所以除

了环境变量，其他uboot、kernel、buildroot也只是重新编译下就可以了。

```
export PATH=$PATH:/home/feixiaoxing/gcc-linaro-7.5.0-2019.12-i686_arm-linux-gnueabihf/bin
```

3、重新编译uboot、测试

首先需要重新编译的就是uboot。之前的配置要记得进行保留，输入make clean就好。接下来就是重新编译make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-。结束之后，再用dd工具烧入到sd卡里面测试下，确保没有问题。

```
make clean
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-
sudo dd if=u-boot-sunxi-with-spl.bin of=/dev/sdb bs=1024 seek=8
```

4、重新编译内核和dtb，结束后烧入测试

uboot测试没有问题之后，同样要对kernel 5.2.y重新进行编译。步骤和uboot差不多，第一步是clean之前生成的中间文件。第二步，重新编译。第三步，分别拷贝dtb和zImage到sd卡，查看是否可以正常启动。

```
make clean
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-

sudo cp /home/feixiaoxing/Desktop/linux-zero-5.2.y/arch/arm/boot/zImage .
sudo cp /home/feixiaoxing/Desktop/linux-zero-5.2.y/arch/arm/boot/dts/sun8i-v3s-licheepi-zero-with-480x272-lcd.dtb .
```

如果一切都没有问题，可以在启动log中看到关于新编译器的打印，

```
Hit any key to stop autoboot:  0
reading zImage
4337216 bytes read in 226 ms (18.3 MiB/s)
reading sun8i-v3s-licheepi-zero-with-480x272-lcd.dtb
11542 bytes read in 26 ms (432.6 KiB/s)
## Flattened Device Tree blob at 41800000
   Booting using the fdt blob at 0x41800000
   Loading Device Tree to 42dfa000, end 42dffd15 ... OK
Cannot setup simplefb: node not found
```

```
Starting kernel ...

[    0.000000] Booting Linux on physical CPU 0x0
[    0.000000] Linux version 5.2.0-licheepi-zero
(feixiaoxing@feixiaoxing-VirtualBox) (gcc version 7.5.0 (Linaro GCC
7.5-2019.12)) #6 SMP Tue Jun 20 22:05:58 CST 2023
[    0.000000] CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7),
cr=10c5387d
[    0.000000] CPU: div instructions available: patching division code
[    0.000000] CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing
instruction cache
[    0.000000] OF: fdt: Machine model: Lichee Pi Zero
[    0.000000] Memory policy: Data cache writealloc
[    0.000000] percpu: Embedded 16 pages/cpu s34508 r8192 d22836 u65536
[    0.000000] Built 1 zonelists, mobility grouping on.  Total pages:
16256
[    0.000000] Kernel command line: console=ttyS0,115200 panic=5
rootwait root=/dev/mmcblk0p2 earlyprintk rw vt.global_cursor_default=0
```

看到gcc version 7.5.0打印后，这个时候就可以确认内核确实使用新编译器编译出来的。

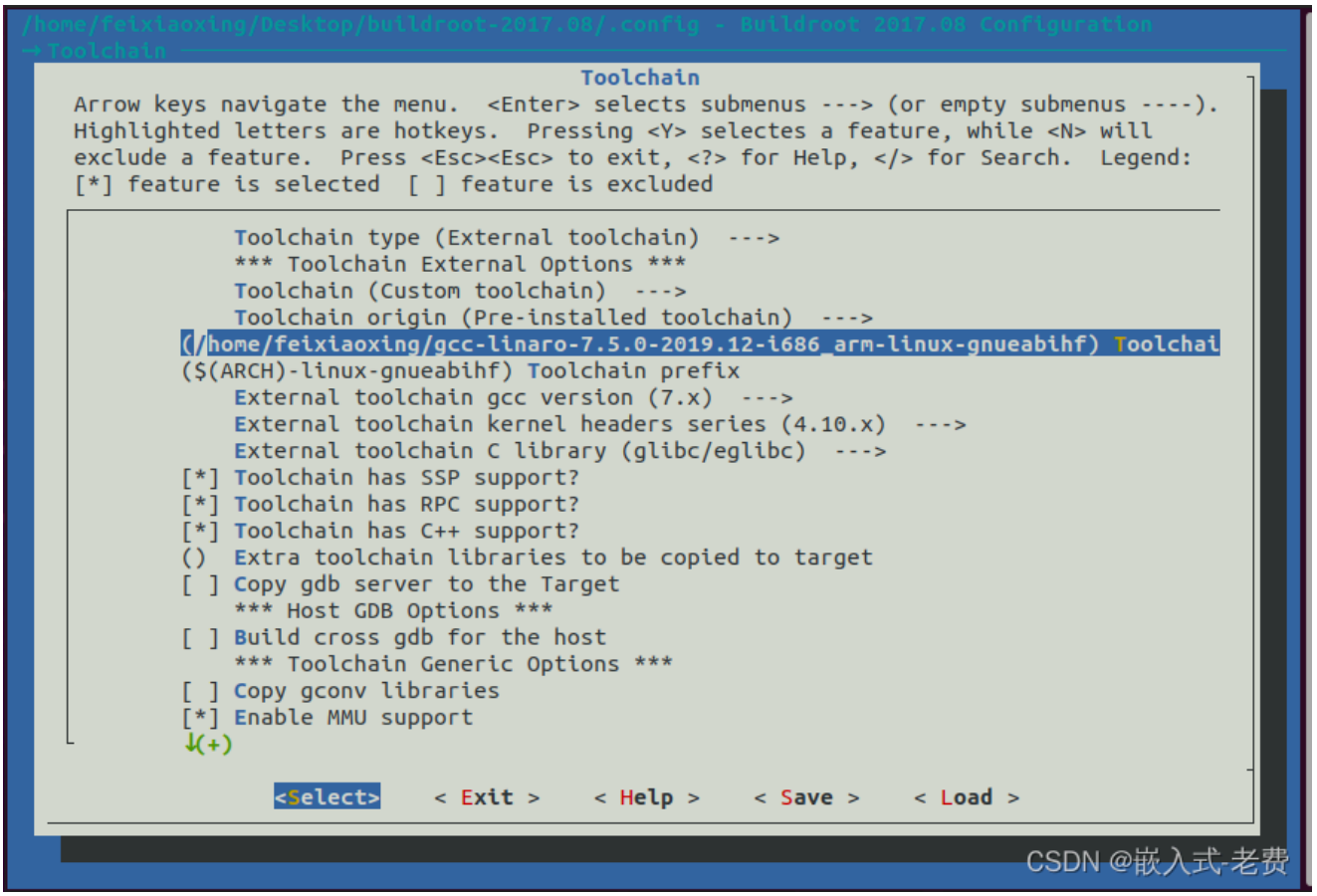
5、重新编译buildroot、测试

完成了uboot和kernel的重新编译之后，就可以开始buildroot的编译。 **要做好buildroot的编译，有两个部分要做好，第一，就是对之前根文件数据要记得保存，毕竟前面做了很多的测试和配置，中间有很多的资料可以参考和借鉴，所以这部分的内容最好做一个备份。**

```
tar -zcvf root_back.tar.gz /media/feixiaoxing/64959927-1d53-4bb1-8102-
d7c16fc29ade/
```

第二，在buildroot中编译器的部分做修改，

Open: 2cbd6210dd441915457336c3763d4650_MD5.png



修改的地方集中在三处，1）新编译器的目录位置；2）新编译器的gcc version修改为7.x；3）kernel headers修改为4.10.x。 **其中最后一条似乎与5.2.y的实际kernel版本不匹配，原因是这个buildroot在2017.8才正式发布的。**

修改完毕，就可以按照uboot和kernel的三部曲开始操作了，分别是clean、make和烧入。其中烧入的话，就是备份完之前的rootfs之后，用新的rootfs来代替。

```
make clean
make
```

接着就是正常的测试验证，如果系统正常起来，就代表ok。

6、用户程序和驱动验证

前面依次完成了uboot、kernel和buildroot的升级。那么在上层程序这部分，用户可以自己编一些程序验证下，也可以查看一下之前的设备驱动是否ok。比如查看lcd背光是否点亮等等，

```
echo 34 > /sys/class/gpio/export
echo out > /sys/class/gpio/gpio34/direction
echo 1 > /sys/class/gpio/gpio34/value
```